oauth2-passkey

Passwordless Authentication Library for Rust

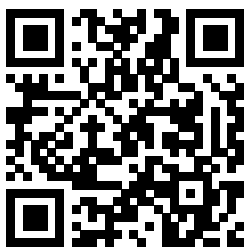
Kimitoshi Takahashi (@ktaka) | Tokyo Rust Show & Tell | 2026/03/31

`crates.io/crates/oauth2-passkey` `crates.io/crates/oauth2-passkey-axum`

Live Demo

Demo

- **Passkey Registration** - Create account with fingerprint/ face
- **Passkey Login** - Authenticate without password
- **Google OAuth2 Login** - Sign in with Google
- **Account Linking** - Connect OAuth2 + Passkey to same user



passkey-demo.ccmp.jp

What the Demo Showed

Feature	Details
OAuth2/OIDC	Google login (also works with FedCM)
Passkey	Google Password Manager, Apple, Windows Hello, bitwarden, Proton Pass, YubiKey
Account Linking	OAuth2 + Passkey mapped to same user
Session	Cookie-based session with CSRF protection

All handled by a single library: `oauth2-passkey`

Motivation

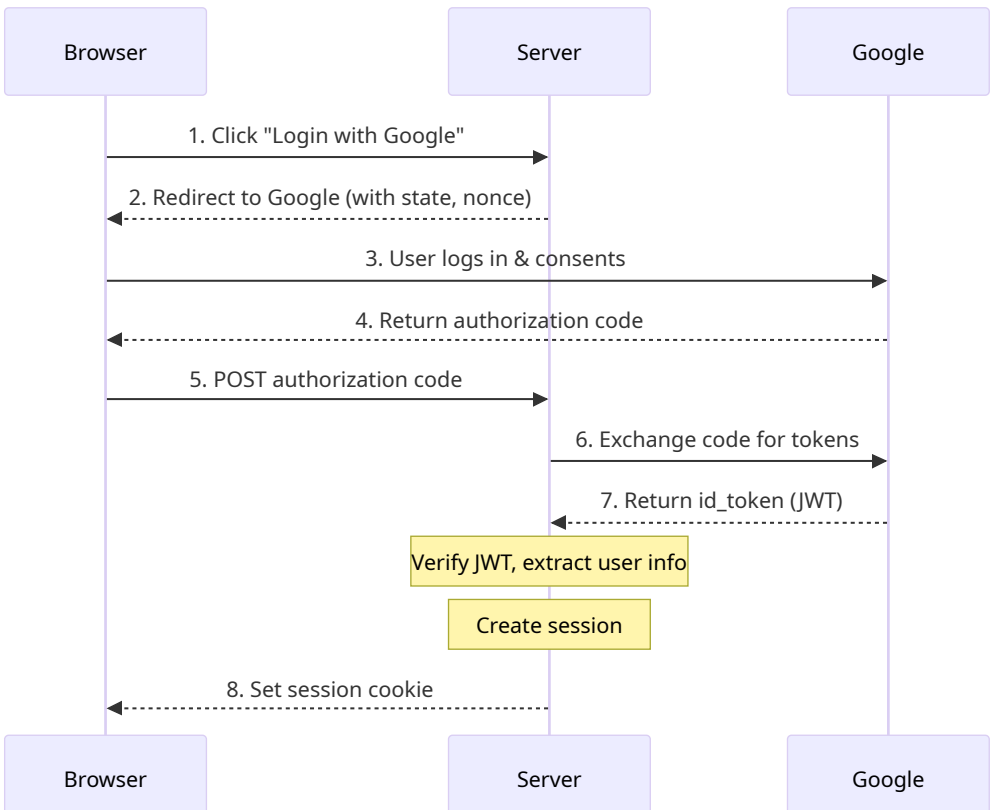
I wanted to build **exactly what you just saw** - and do it myself in Rust.

- Existing Rust ecosystem:
  - `webauthn-rs` - WebAuthn only
  - Various OAuth2 crates - OAuth2 only
  - **No combined solution** with session management
- So I built one. Published on **crates.io**.

Agenda

1. **How OAuth2 & Passkey Work -** What happened behind the demo
2. **Using the Library -** init, extractor, middleware
3. **Storage & LazyLock -** Multi-DB support, why not Axum State
4. **Integrating with Your App -** Linking your data to auth users
5. **Wrap-up -** Summary & links

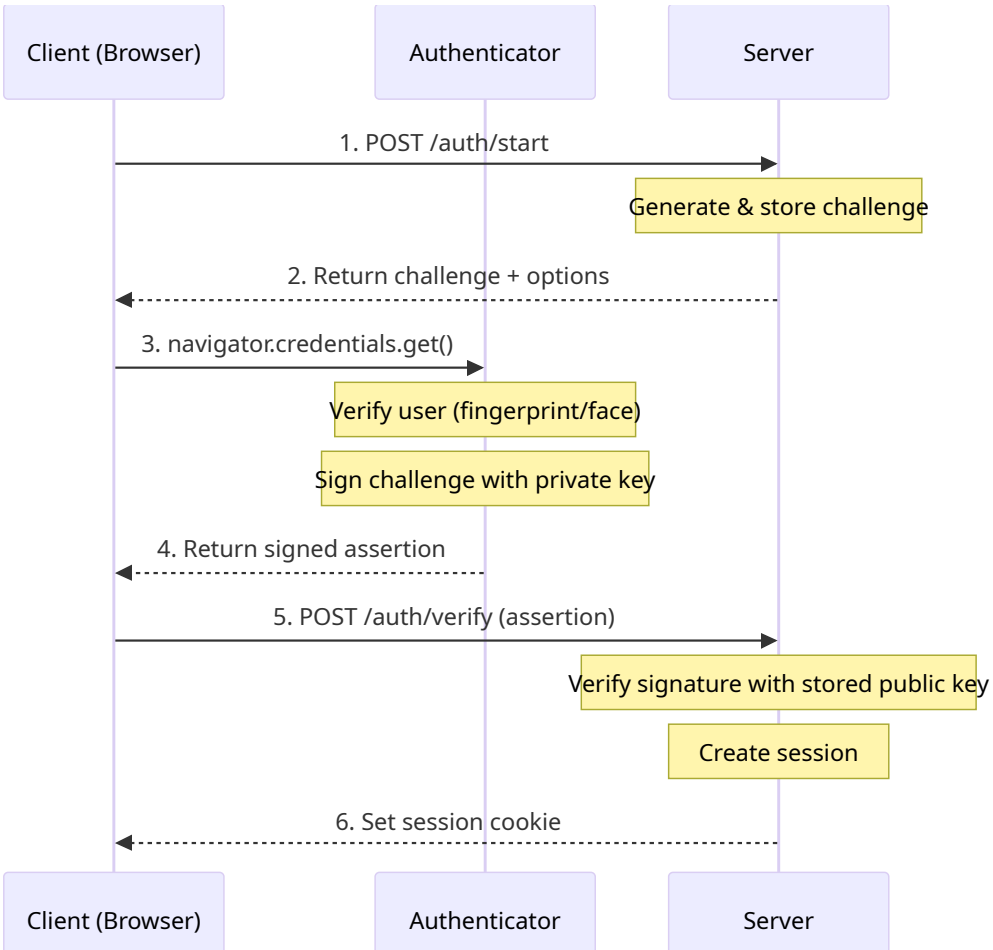
How OAuth2/OIDC Works



Page-redirect based auth:

1. User clicks “Login with Google”
2. Redirect to Google consent screen
3. Google returns authorization code
4. Server exchanges code for **id_token** (JWT)
5. Extract user info, create session
6. Set session cookie

How Passkey/WebAuthn Works



JavaScript-driven, no redirects:

1. Server generates **challenge**
2. Browser calls `navigator.credentials.get()`
3. Authenticator signs challenge with **private key**
4. Server verifies with stored **public key**
5. Create session, set cookie

Authenticators: Google Password Manager, YubiKey, Touch ID, Windows Hello

Using the Library

Setup: init + merge

```rust
use oauth2_passkey_axum::{
    AuthUser, oauth2_passkey_full_router,
};

#[tokio::main]
async fn main() -> Result<(), Box<dyn std::error::Error>> {
    dotenv().ok();
    oauth2_passkey_axum::init().await?;       // 1. Initialize

    let app = Router::new()
        .route("/", get(index))
        .merge(oauth2_passkey_full_router()); // 2. Merge router

    // Auth endpoints are now at /o2p/*
    spawn_http_server(3001, app).await?;
    Ok(())
}
```

Page Protection: AuthUser Extractor

```rust
// Required auth - redirects to login if not authenticated
async fn protected(user: AuthUser) -> impl IntoResponse {
    format!("Hello, {}!", user.label)
}

// Optional auth - allows anonymous access
async fn public(user: Option<AuthUser>) -> impl IntoResponse {
    match user {
        Some(u) => format!("Hello, {}!", u.label),
        None    => "Hello, anonymous!".to_string(),
    }
}
```

Implemented via Axum's `FromRequestParts` trait:

- `AuthUser` -> auto-redirect on failure
- `Option<AuthUser>` -> no redirect, `None` for anonymous

Page Protection: Middleware

```
let app = Router::new()
    // Protect single route
    .route("/secret", get(handler)
        .route_layer(from_fn(
            is_authenticated_redirect)))
    // Protect entire group
    .nest("/api", api_router()
        .route_layer(from_fn(
is_authenticated_user_redirect)));
```

Middleware	Unauth	User?
`_401`	401	No
`_redirect`	Login	No
`_user_401`	401	Yes
`_user_redirect`	Login	Yes

All prefixed with `is_authenticated`

Storage & LazyLock Pattern

Multi-DB Support with sqlx

DataStore trait:

```rust
pub(crate) trait DataStore: Send + Sync {
    fn as_sqlite(&self)
        -> Option<&Pool<Sqlite>>;
    fn as_postgres(&self)
        -> Option<&Pool<Postgres>>;
    fn as_mysql(&self)
        -> Option<&Pool<MySql>>;
}
```

One trait, three implementations. No runtime downcasting needed.

Dispatch pattern:

```rust
let store = GENERIC_DATA_STORE
    .lock().await;

match (store.as_sqlite(),
       store.as_postgres(),
       store.as_mysql()) {
    (Some(pool), _, _) =>
        do_sqlite(pool).await?,
    (_, Some(pool), _) =>
        do_postgres(pool).await?,
    (_, _, Some(pool)) =>
        do_mysql(pool).await?,
    _ => return Err(...),
}
```

Why LazyLock Instead of Axum State?

Typical Axum State pattern:

```
struct AppState { db: PgPool, cache:
Redis }
let app =
Router::new().with_state(state);

// EVERY handler needs State
async fn handler(
    State(s): State<AppState>
) { ... }

// 80+ internal functions need &AppState
async fn internal_fn(
    state: &AppState
) { ... }
```

oauth2-passkey: LazyLock globals

```
static GENERIC_DATA_STORE:
    LazyLock<Mutex<Box<dyn DataStore>>>
    = LazyLock::new(|| {
        match store_type {
            "sqlite"   => Box::new(...),
            "postgres" => Box::new(...),
            "mysql"    => Box::new(...),
        }
    });

// Any function can access directly
let store = GENERIC_DATA_STORE
    .lock().await;
```

LazyLock: Benefits & Trade-offs

Benefits

- **Zero boilerplate for users** - no `AppState` struct to create
- **No state threading** - internal functions access storage directly
- **Env-only config** - set `GENERIC_DATA_STORE_TYPE=postgres` in `.env`
- **Fail-fast init** - `init().await?` forces evaluation at startup

Trade-offs

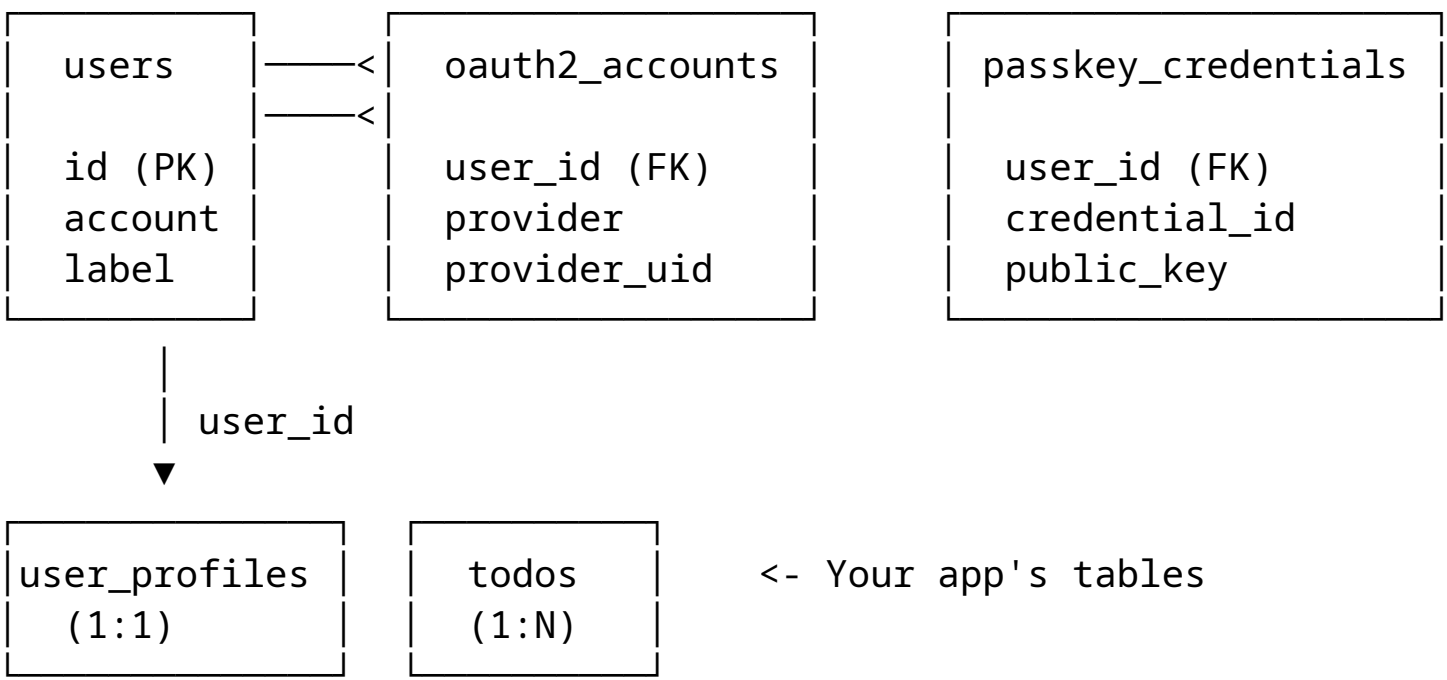
- Single instance per process (fine for auth library)
- Implicit dependencies (function signatures don’t show DB access)
- Test isolation needs `#[serial]` (shared global state)

A library that requires users to manage `AppState` is harder to adopt. LazyLock keeps complexity **inside** the library.

Integrating with Your App

Account Linking: Internal DB Structure

oauth2-passkey manages these tables:



One user can have multiple OAuth2 accounts AND multiple passkeys.

Your App's Data: Link via AuthUser.id

Setup: two databases, one app

```
// 1. oauth2-passkey init
oauth2_passkey_axum::init().await?;

// 2. App's own DB
let pool = db::init_db().await?;
let state = AppState { pool };

// 3. Combine routes
let app = Router::new()
    .route("/", get(index))
    .merge(handlers::router())
    .with_state(state)
    .merge(oauth2_passkey_full_router());
```

Handler: use user.id as FK

```
async fn create_todo(
    State(state): State<AppState>,
    user: AuthUser,
    Form(form): Form<TodoForm>,
) -> Result<Response, ...> {
    db::create_todo(
        &state.pool,
        &user.id,  // link to auth user
        &form.title,
    ).await?;
    Ok(Redirect::to("/").into_response())
}
```

See `demo-profile` (1:1) and `demo-todo` (1:N).

Wrap-up

Summary

What	Details
Library	`oauth2-passkey` + `oauth2-passkey-axum`
What it does	Passkey + OAuth2 auth for Axum apps
DB support	SQLite, PostgreSQL, MySQL (via sqlx)
Key design	LazyLock globals, DataStore trait dispatch
Integration	`AuthUser.id` links your app data to auth

GitHub

Links

- **crates.io:** `oauth2-passkey`, `oauth2-passkey-axum`
- **GitHub:** github.com/ktaka-ccmp/oauth2-passkey
- **X:** @ktaka

Thank You!

Questions?

Kimitoshi Takahashi (@ktaka) on X / GitHub

About Me

- **@ktaka** (X / GitHub)
- Self-employed, reskilling in Rust
- Building web authentication libraries
- Third year writing Rust

ktaka.blog.ccmp.jp/p/p.html

Contact